

Procesarea și conversia datelor din fișiere, validare și automatizări practice

Paul A. Gagniuc

Cuprins

- ✓ Plotare date din CSV
- ✓ Date din CSV și grafic liniar
- ✓ Integrare plot în interfață Tkinter
- ✓ Validare input din Entry
- ✓ Legarea butonului Check de tasta Enter
- ✓ Numere cu virgulă și calcul smooth
- ✓ Citire CSV mare fără încărcare completă
- ✓ Verificare date lipsă și duplicate
- ✓ Comparare două fișiere CSV
- ✓ Export SQL mare în fișiere Excel mai mici
- ✓ Raport statistic pe regiuni și tipuri de clienți
- ✓ Citire toate taburile dintr-un Excel
- ✓ Citire fișier XBRL/XML și export în Excel + CSV
- ✓ Conversie Excel → XML/XBRL simplificat
- ✓ Reconciliere între sursă și țintă
- ✓ Reguli de consistență pentru clienți
- ✓ Event Notifier simplificat
- ✓ Count down



Plotare Date din CSV

C: > Users > Paul > Desktop > ex.py > ...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 df = pd.read_csv("vanzari.csv")
5
6 plt.plot(df["luna"], df["vanzari"])
7 plt.xlabel("Luna")
8 plt.ylabel("Vanzari")
9 plt.title("Vanzari din CSV")
10 plt.show()
```

pandas

Adaptati calea!

Date din CSV + line plot
CSV vanzari.csv:

```
luna,vanzari
Ian,100
Feb,120
Mar,115
Apr,140
Mai,160
```



```
import tkinter as tk
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

def deseneaza_plot():
    ax.clear()
    ax.plot(x, y)
    ax.set_xlabel("X")
    ax.set_ylabel("Y")
    ax.set_title("Line plot")
    canvas.draw()

x = [1, 2, 3, 4, 5]
y = [10, 14, 13, 18, 20]

root = tk.Tk()
root.title("Plot in aceeași fereastră")
root.geometry("700x500")

buton = tk.Button(root, text="Afiseaza plot", command=deseneaza_plot)
buton.pack(pady=10)

fig = Figure(figsize=(6, 4), dpi=100)
ax = fig.add_subplot(111)

canvas = FigureCanvasTkAgg(fig, master=root)
canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

root.mainloop()
```





Validare input din
Entry

```
import tkinter as tk
from tkinter import messagebox

def verifica():
    text = entry.get()

    try:
        valoare = float(text.replace(",", "."))
        label_rezultat.config(text=f"Valoare validă: {valoare}")
    except ValueError:
        messagebox.showerror("Eroare", "Introduceți un număr valid.")

root = tk.Tk()
root.title("Validare input")
root.geometry("300x150")

entry = tk.Entry(root)
entry.pack(pady=10)

button = tk.Button(root, text="Check", command=verifica)
button.pack()

label_rezultat = tk.Label(root, text="")
label_rezultat.pack(pady=10)

root.mainloop()
```





Legarea butonului
Check de tasta Enter


```
root.bind("<Return>", lambda event: verifica())
```

Numere cu virgulă și
calcul smooth

```
import tkinter as tk
from tkinter import messagebox

def calculeaza():
    text = entry.get()

    try:
        valori = [float(x.strip().replace(",", ".")) for x in text.split(";")]
        valori_smooth = smooth(valori, 3)
        label.config(text=str(valori_smooth))
    except ValueError:
        messagebox.showerror("Eroare", "Introduceți valori separate prin ;")

def smooth(lista, fereastră=3):
    rezultat = []

    for i in range(len(lista)):
        start = max(0, i - fereastră + 1)
        valori = lista[start:i+1]
        rezultat.append(sum(valori) / len(valori))

    return rezultat
```



```
root = tk.Tk()
root.geometry("500x200")
root.title("Smooth numeric")

entry = tk.Entry(root, width=60)
entry.pack(pady=10)

button = tk.Button(root, text="Calculează smooth", command=calculeaza)
button.pack()

label = tk.Label(root, text="")
label.pack(pady=10)

root.mainloop()
```



Citire CSV mare fără
încărcare completă

```
import pandas as pd
```

```
fisier = "date_mari.csv"
```

```
total = 0
```

```
for bucata in pd.read_csv(fisier, chunksize=1000):  
    total += bucata["valoare"].sum()
```

```
print("Total:", total)
```



Citiți un fișier CSV mare pe bucăți și calculați totalul unei coloane.

Verificare date lipsă și
duplicate

```
import pandas as pd

df = pd.read_csv("clienti.csv")

print("Valori lipsă:")
print(df.isna().sum())

print("Duplicate:")
print(df.duplicated().sum())

df_curat = df.dropna()
df_curat = df_curat.drop_duplicates()

print("Dimensiune inițială:", df.shape)
print("Dimensiune după curățare:", df_curat.shape)

df_curat.to_csv("clienti_curat.csv", index=False)
```



Comparare două fișiere CSV

```
import pandas as pd
```

```
df1 = pd.read_csv("fisier1.csv")
```

```
df2 = pd.read_csv("fisier2.csv")
```

```
diferente = df1.merge(df2, how="outer", indicator=True)
```

```
diferente = diferente[diferente["_merge"] != "both"]
```

```
print(diferente)
```

```
diferente.to_csv("diferente.csv", index=False)
```

Comparați două fișiere CSV și
afișați liniile care există în primul,
dar nu există în al doilea.

păstrează doar rândurile care nu sunt în ambele fișiere.

`diferente["_merge"] != "both"`

both = există în ambele fișiere
left_only = există doar în primul
right_only = există doar în al doilea

Deci _merge este doar o etichetă pentru comparație.

Export SQL mare în
fișiere Excel mai mici

Împarte două numere și păstrează doar partea întreagă.

```
import pandas as pd

df = pd.read_csv("export_sql.csv")

marime = 1000

for i in range(0, len(df), marime):
    bucata = df.iloc[i:i+marime]
    nume_fisier = f"export_partea_{i//marime + 1}.xlsx"
    bucata.to_excel(nume_fisier, index=False)

print("Export finalizat.")
```

iloc înseamnă index location.

Simulați un export mare și împărțiți datele în mai multe fișiere Excel de câte 1000 de linii.



Raport statistic pe regiuni și tipuri de clienți

Dintr-un fișier cu produse, regiuni, vârste și tipuri de clienți, calculați numărul de produse pe regiune și pe tip de client.

```
import pandas as pd

df = pd.read_csv("produse.csv")

raport = df.groupby(["regiune", "tip_client"])["produs"].count()

print(raport)

raport.to_excel("raport_statistic.xlsx")
```

produs	regiune	tip_client	varsta
Laptop	Bucuresti	Individual	34
Telefon	Cluj	Individual	22
Tableta	Bucuresti	Companie	41
Monitor	Iasi	Companie	38
Mouse	Cluj	Individual	29
Tastatura	Iasi	Individual	25
Laptop	Bucuresti	Companie	45
Telefon	Timisoara	Individual	31
Monitor	Timisoara	Companie	52
Tableta	Cluj	Companie	40
Mouse	Bucuresti	Individual	27
Tastatura	Iasi	Companie	36
Laptop	Timisoara	Individual	33
Telefon	Bucuresti	Individual	24
Monitor	Cluj	Companie	48





Citire toate taburile
dintr-un Excel

```
import pandas as pd
```

```
fisier = "date.xlsx"
```

```
toate_foile = pd.read_excel(fisier, sheet_name=None)
```

```
for nume_foiaie, tabel in toate_foile.items():  
    print("Foaie:", nume_foiaie)  
    print(tabel.head())
```


Citire fișier XBRL/XML
și export în Excel +
CSV

C: > Users > Paul > Desktop > ex.py > ...

```
1  import os
2  import xml.etree.ElementTree as ET
3  import pandas as pd
4
5  cale_fisier = r"C:\Users\Paul\Desktop\raport.xml"
6
7  if not os.path.exists(cale_fisier):
8      print("Eroare: fișierul nu există:")
9      print(cale_fisier)
10     exit()
11
12  tree = ET.parse(cale_fisier)
13  root = tree.getroot()
14
15  date = []
16
17  for element in root.findall("item"):
18      date.append({
19          "indicator": element.attrib.get("name"),
20          "valoare": element.text
21      })
22
23  df = pd.DataFrame(date)
24
25  print(df)
26
27  df.to_excel(r"C:\Users\Paul\Desktop\raport_excel.xlsx", index=False)
28  df.to_csv(r"C:\Users\Paul\Desktop\raport_csv.csv", index=False)
29
30  print("Export terminat.")
```

Exemplu simplu de raport.xml

```
<raport>
<item name="Active">1000000</item>
<item name="Datorii">400000</item>
<item name="Capital">600000</item>
</raport>
```

Conversie Excel →
XML/XBRL simplificat

C: > Users > Paul > Desktop > ex.py > ...

```
1  import pandas as pd
2  import xml.etree.ElementTree as ET
3
4  # Citire Excel
5  df = pd.read_excel(r"C:\Users\Paul\Desktop\date.xlsx")
6
7  # Creare structură XML
8  raport = ET.Element("raport")
9
10 for index, row in df.iterrows():
11     item = ET.SubElement(raport, "item")
12
13     item.set("name", str(row["indicator"]))
14
15     item.text = str(row["valoare"])
16
17
18 # Salvare XML
19 tree = ET.ElementTree(raport)
20
21 tree.write(
22     r"C:\Users\Paul\Desktop\fisier_generat.xml",
23     encoding="utf-8",
24     xml_declaration=True
25 )
26
27 print("Fișier XML generat.")
```

Trebuie sa avem un fisier date.xlsx pe Desktop



Reconciliere între
sursă și țintă

C: > Users > Paul > Desktop > ex.py > ...

```
1 import pandas as pd
2
3 # Fișier sursă
4 sursa = pd.read_csv(r"C:\Users\Paul\Desktop\sursa.csv")
5
6 # Fișier țintă
7 tinta = pd.read_csv(r"C:\Users\Paul\Desktop\tinta.csv")
8
9 # Comparare
10 comparatie = sursa.merge(
11     tinta,
12     on="cont",
13     suffixes=("_sursa", "_tinta")
14 )
15
16 # Diferență
17 comparatie["diferenta"] = (
18     comparatie["sold_sursa"] -
19     comparatie["sold_tinta"]
20 )
21
22 # Linii cu probleme
23 erori = comparatie[
24     comparatie["diferenta"] != 0
25 ]
26
27 print(erori)
28
29 # Salvare raport
30 erori.to_excel(
31     r"C:\Users\Paul\Desktop\raport_diferente.xlsx",
32     index=False
33 )
34
35 print("Raport generat.")
```

Compară două fișiere CSV, sursa.csv și tinta.csv, după coloana cont, apoi verifică dacă soldurile sunt egale.

sursa.csv

tinta.csv

cont,sold

cont,sold

A1,1000

A1,1000

A2,2500

A2,2400

A3,3000

A3,3000

Reguli de consistență
pentru clienți

C: > Users > Paul > Desktop > ex.py > ...

```
1 import pandas as pd
2
3 df = pd.read_excel(
4     r"C:\Users\Paul\Desktop\clienti.xlsx"
5 )
6
7 erori = []
8
9 for index, row in df.iterrows():
10
11     if row["venit"] <= 0:
12         erori.append([
13             row["client"],
14             "Venitul trebuie să fie pozitiv"
15         ])
16
17     if row["datorie"] < 0:
18         erori.append([
19             row["client"],
20             "Datoria nu poate fi negativă"
21         ])
22
23     if row["varsta"] < 18 or row["varsta"] > 100:
24         erori.append([
25             row["client"],
26             "Vârsta invalidă"
27         ])
28
29 raport_erori = pd.DataFrame(
30     erori,
31     columns=["client", "eroare"]
32 )
33
34 print(raport_erori)
35
```

```
36 raport_erori.to_excel(
37     r"C:\Users\Paul\Desktop\erori_validare.xlsx",
38     index=False
39 )
40
41 print("Validare terminată.")
```

	A	B	C	D	E
1	client	venit	datorie	varsta	
2	C1	5000	1000	34	
3	C2	-1000	2000	17	
4	C3	3000	-500	45	
5	C4	4000	6000	200	
6					
7					

clienti.xlsx



```
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Paul\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Paul/Desktop/ex.py
client      eroare
0    C2  Venitul trebuie să fie pozitiv
1    C2    Vârsta invalidă
2    C3  Datoria nu poate fi negativă
3    C4    Vârsta invalidă
Validare terminată.
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code>
```




Event Notifier simplificat

C: > Users > Paul > Desktop > ex.py > ...

```
1  import time
2  from datetime import datetime
3
4  evenimente = [
5      {
6          "titlu": "Raport zilnic",
7          "ora": 12,
8          "minut": 3,
9          "mesaj": "Verifică raportul bancar."
10     }
11 ]
12
13 notificat = False
14
15 while True:
16     acum = datetime.now()
17
18     print("Ora curentă:", acum.strftime("%H:%M:%S"))
19
20     for eveniment in evenimente:
21         if (
22             acum.hour == eveniment["ora"] and
23             acum.minute == eveniment["minut"] and
24             notificat == False
25         ):
26             print("NOTIFICARE")
27             print(eveniment["titlu"])
28             print(eveniment["mesaj"])
29
30             notificat = True
31
32         if acum.minute != evenimente[0]["minut"]:
33             notificat = False
34
35     time.sleep(1)
```





Count down

```
1  import time
2  from datetime import datetime, timedelta
3
4  evenimente = [
5      {
6          "titlu": "Raport zilnic",
7          "ora": 12,
8          "minut": 30,
9          "mesaj": "Verifică raportul bancar."
10     }
11 ]
12
13 notificat = False
14
15 while True:
16
17     acum = datetime.now()
18
19     tinta = acum.replace(
20         hour=evenimente[0]["ora"],
21         minute=evenimente[0]["minut"],
22         second=0,
23         microsecond=0
24     )
25
26     if tinta < acum:
27         tinta = tinta + timedelta(days=1)
28
29     diferenta = tinta - acum
30
31     # elimină miimile / microsecundele
32     diferenta = diferenta - timedelta(microseconds=diferenta.microseconds)
33
34     print("Timp rămas până la notificare:", diferenta)
35
36     for eveniment in evenimente:
37
38         if (
39             acum.hour == eveniment["ora"] and
40             acum.minute == eveniment["minut"] and
41             notificat == False
42         ):
```



Ză end!

